

Spring

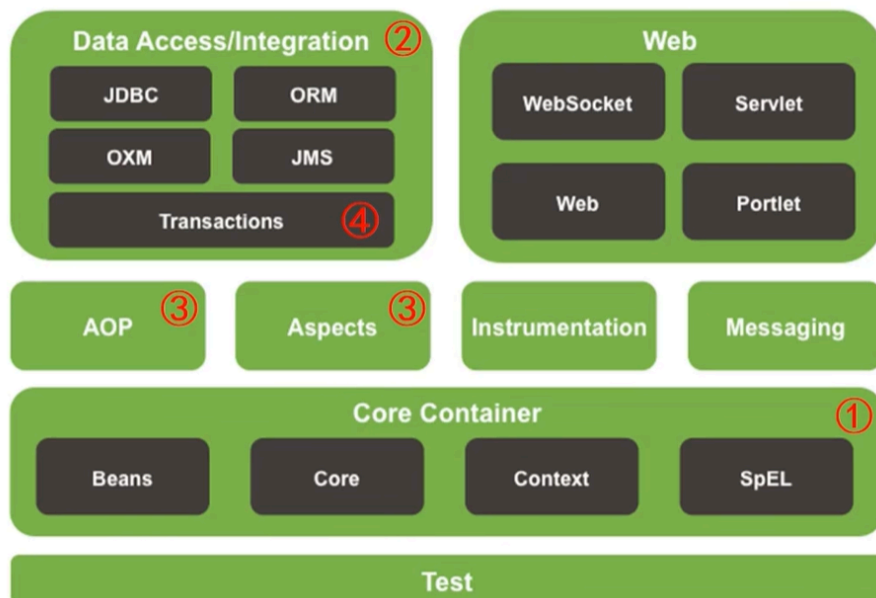
鸣谢：黑马程序员



黑马程序员SSM框架教程_Spring+SpringMVC+Maven高级+SpringBo...

UP 黑马程序员 · 2022-4-20

Spring Framework系统架构



- Data Access: 数据访问
- Data Integration: 数据集成
- Web: Web开发
- AOP: 面向切面编程
- Aspects: AOP思想实现
- Core Container: 核心容器
- Test: 单元测试与集成测试

一、核心容器

1. IoC/DI

1.1 IoC

- `IoC(Inversion of Control)`：控制反转。
- **核心思想**：将对象的创建控制权由程序内部转移到**外部**。也就是使用对象时，在程序中不要主动使用 `new` 产生对象，而是转换为由外部提供对象。

1.2 IoC 容器

- Spring提供了一个容器用来充当 `IoC` 思想中的**外部**，称为 `IoC` 容器。

1.3 Bean

📢 Important

`IoC` 容器负责对象的创建、初始化等一系列工作，被创建或被管理的对象在 `IoC` 容器中统称为 `Bean`。

P1 `Bean` 的作用范围(`scope`)

- 默认是单例。
- 适合交给容器管理的 `Bean`：接口层对象、业务层对象、数据层对象、工具对象；
- 不适合交给容器管理的 `Bean`：封装实体的域对象。

P2 实例化 `Bean` 的方式

- **常用**：提供无参构造器（`private` 也可）。
- 使用静态工厂。
- 使用实例工厂。

- **重点：**使用 `FactoryBean`。（方式三的变种）

P3 `Bean` 的生命周期及控制

- `Bean` 的生命周期：
 - **Step1: 初始化容器。**
 1. 创建对象（内存分配）
 2. 执行构造器
 3. 执行属性注入（调用 `setter`）
 4. 执行 `bean` 初始化方法
 - **Step2: 使用 `bean`。**
 - **Step3: 关闭/销毁容器。**
- `Bean` 的生命周期控制：
 - 控制方式一：

- 提供生命周期控制方法

```
public class BookDaoImpl implements BookDao {  
    public void save() {  
        System.out.println("book dao save ...");  
    }  
    public void init(){  
        System.out.println("book init ...");  
    }  
    public void destory(){  
        System.out.println("book destory ...");  
    }  
}
```

- 配置生命周期控制方法

```
<bean id="bookDao" class="com.itheima.dao.impl.BookDaoImpl" init-method="init" destroy-method="destory"/>
```

- 控制方式二：接口控制

- 实现InitializingBean, DisposableBean接口

```
public class BookServiceImpl implements BookService , InitializingBean, DisposableBean {
    public void save() {
        System.out.println("book service save ...");
    }
    public void afterPropertiesSet() throws Exception {
        System.out.println("afterPropertiesSet");
    }
    public void destroy() throws Exception {
        System.out.println("destroy");
    }
}
```

P4 Bean 的销毁时机

- 容器关闭前触发 Bean 的销毁。
- 关闭容器的方式：
 - 手动关闭：调用 ConfigurableApplicationContext 接口的 close() 方法。
 - 注册关闭钩子，先关闭容器再退出JVM：调用 ConfigurableApplicationContext 接口的 registerShutdownHook() 方法。

1.4 DI

P1 概述

- DI(Dependency Injection)：依赖注入。
- 核心思想：在 IoC 容器中建立 Bean 与 Bean 之间的依赖关系。

业务层实现

```
public class BookServiceImpl implements BookService {
    private BookDao bookDao;

    public void save() {
        bookDao.save();
    }
}
```

依赖dao对象运行

数据层实现

```
public class BookDaoImpl implements BookDao {
    public void save() {
        System.out.println("book dao save ...");
    }
}
```

IoC容器



- 向一个类中传递的数据类型：
 - 简单类型：基本数据类型 + String --> <value>

- 引用类型--> `<ref>`

P2 依赖注入方式

- **setter 注入：**
 - 简单类型
 - 引用类型
- **构造器注入：**
 - 简单类型
 - 引用类型

P3 如何选择依赖注入方式？

- 具有强制依赖关系的使用构造器注入，因为使用 `setter` 注入有概率没有进行注入，导致 `null` 对象出现。
- 具有可选依赖关系的使用 `setter` 注入，灵活性强。
- Spring框架推荐使用构造器注入，第三方框架内部大多采用构造器注入的形式进行数据初始化，相对严谨。
- 如有必要可以两者同时使用：
 - 使用构造器注入完成强制依赖关系的注入；
 - 使用 `setter` 注入完成可选依赖关系的注入。
- 实际开发过程中根据实际情况分析，如果受控对象没有提供 `setter`，则必须使用构造器注入。
- 自己开发的模块**推荐**使用 `setter` 注入。

P4 依赖自动装配

- **含义：** `IoC` 容器根据 `bean` 所依赖的资源在容器中自动查找并注入到 `bean` 中。
- **自动装配方式：**
 - 按类型（常用）
 - 按名称

- 按构造器
- 不启用自动装配

- **注意事项：**

- 自动装配仅用于引用类型的依赖注入，无法对简单类型进行依赖注入。
- 使用按类型装配(`byType`)时，必须保证容器中相同类型的 `bean` 只有一个，开发中**推荐**使用这种方式进行自动装配。
- 使用按名称装配(`byName`)时，必须保证容器中具有指定名称的 `bean`（即 `setter` 方法的名字去掉 `set` 后的名称），因为变量名与配置高度耦合，故不推荐使用。
- 自动装配的优先级低于 `setter` 注入和构造器注入，同时出现时自动装配失效。

P5 集合注入

- `BookDaoImpl`：

```
1 package com.zsh.dao.impl;
2
3 import com.zsh.dao.BookDao;
4 import java.util.*;
5
6 public class BookDaoImpl implements BookDao {
7
8     private int[] zshArray;
9     private List<String> zshList;
10    private Set<String> zshSet;
11    private Map<String,String> zshMap;
12    private Properties zshProperties;
13
14    public void setZshArray(int[] zshArray) {
15        this.zshArray = zshArray;
16    }
17    public void setZshList(List<String> zshList) {
18        this.zshList = zshList;
19    }
20    public void setZshSet(Set<String> zshSet) {
21        this.zshSet = zshSet;
22    }
23    public void setZshMap(Map<String, String> zshMap) {
24        this.zshMap = zshMap;
25    }
26 }
```

```

26     public void setZshProperties(Properties zshProperties) {
27         this.zshProperties = zshProperties;
28     }
29
30     @Override
31     public void save() {
32         System.out.println("book dao save ...");
33         System.out.println("traverse Array: "+
Arrays.toString(zshArray));
34         System.out.println("traverse List: "+ zshList);
35         System.out.println("traverse Set: "+ zshSet);
36         System.out.println("traverse Map: "+ zshMap);
37         System.out.println("traverse Properties: "+ zshProperties);
38     }
39 }

```

- applicationContext.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <beans xmlns="http://www.springframework.org/schema/beans"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
5
6      <bean id="bookDao" class="com.zsh.dao.impl.BookDaoImpl">
7          <property name="zshArray">
8              <array>
9                  <value>100</value>
10                 <value>200</value>
11                 <value>300</value>
12             </array>
13         </property>
14
15         <property name="zshList">
16             <list>
17                 <value>zsh</value>
18                 <value>zjl</value>
19                 <value>zxj</value>
20                 <value>zxj</value>
21             </list>
22         </property>
23
24         <property name="zshSet">
25             <set>

```

```
26         <value>apple</value>
27         <value>banana</value>
28         <value>strawberry</value>
29         <value>strawberry</value>
30         <!-- 由于Set元素不可重复，故会自动过滤2个重复的strawberry-->
31     </set>
32 </property>
33
34 <property name="zshMap">
35     <map>
36         <entry key="country" value="China"/>
37         <entry key="province" value="Guangdong"/>
38         <entry key="city" value="Swatow"/>
39     </map>
40 </property>
41
42 <property name="zshProperties">
43     <props>
44         <prop key="name">zsh</prop>
45         <prop key="sex">male</prop>
46         <prop key="height">175</prop>
47     </props>
48 </property>
49 </bean>
50
51 </beans>
```


P6 加载 properties 文件

加载properties文件

- 开启context命名空间

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context.xsd">
</beans>
```

- 使用context命名空间，加载指定properties文件

```
<context:property-placeholder location="jdbc.properties"/>
```

- 使用\${}读取加载的属性值

```
<property name="username" value="${jdbc.username}"/>
```

- 不加载系统属性

```
<context:property-placeholder location="jdbc.properties" system-properties-mode="NEVER"/>
```

- 加载多个properties文件

```
<context:property-placeholder location="jdbc.properties,msg.properties"/>
```

- 加载所有properties文件

```
<context:property-placeholder location="*.properties"/>
```

- 加载properties文件**标准格式**

```
<context:property-placeholder location="classpath:*.properties"/>
```

- 从类路径或jar包中搜索并加载properties文件

```
<context:property-placeholder location="classpath*:*.properties"/>
```

2.容器基本操作

2.1 创建容器

2.2 获取 `bean`

2.3 容器类层次结构

2.4 `BeanFactory`

二、数据访问与集成

三、AOP

四、事务

五、框架整合